

REACT HOOK FORM + ZOD - COMPLETE NOTES

Page

1. What is React Hook Form ?

- React Hook Form (RHF) is a performant, flexible and lightweight library for building forms in React.
- Uses Uncontrolled Components + Refs internally for better performance.

Less Re-render
Better Performance
Easy Validation
DX



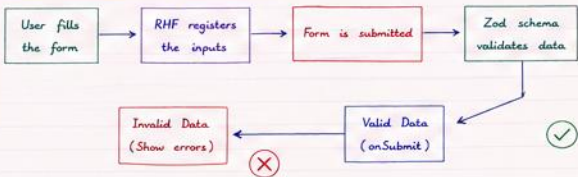
2. What is Zod ?

- Zod is a TypeScript-first schema validation library.
- Used to define schema and validate data.
- Works great with React Hook Form via [@hookform/resolvers/zod](https://github.com/hookform/resolvers/zod)

3. Installation

```
npm install react-hook-form @hookform/resolvers zod
```

4. How it Works ? (High Level Flow)



5. Basic Setup (Form with Zod)

Step 1: Import

```
import { useForm } from 'react-hook-form'
import { z } from 'zod'
import { zodResolver } from '@hookform/resolvers/zod'
```

Step 2: Create Zod Schema

```
const schema = z.object({
  name: z.string().min(2, 'Name must be at least 2 characters'),
  email: z.string().email('Invalid email address'),
  age: z.coerce.number().min(18, 'Must be 18 or above'),
  password: z.string().min(6, 'Password must be at least 6 chars'),
  confirm: z.string()
}).refine((data) => data.password === data.confirm, {
  message: 'Passwords do not match',
  path: ['confirm']
})
```

.min() =>
Minimum length
.email() =>
Valid email
.refine() =>
Custom validation

6. Use useForm() with zodResolver

```
const { register, handleSubmit, formState: { errors } } = useForm({
  resolver: zodResolver(schema)
})
```

resolver connects
Zod schema
with RHF

7. Register Inputs

```

<input type="text" {...register('name')} placeholder="Enter name" />
{errors.name && <p className="error">{errors.name.message}</p>}

<input type="email" {...register('email')} placeholder="Enter email" />
{errors.email && <p className="error">{errors.email.message}</p>}

<input type="number" {...register('age')} placeholder="Enter age" />
{errors.age && <p className="error">{errors.age.message}</p>}

<input type="password" {...register('password')} placeholder="Password" />
{errors.password && <p className="error">{errors.password.message}</p>}

<input type="password" {...register('confirm')} placeholder="Confirm Password" />
{errors.confirm && <p className="error">{errors.confirm.message}</p>}

```

errors
contains
validation
messages
from Zod

8. Handle Submit

```

const onSubmit = (data) => {
  console.log('Form Data.', data)
}

<form onSubmit={handleSubmit(onSubmit)} noValidate>
  {/* Inputs */}
  <button type="submit">Submit</button>
</form>

```

handleSubmit
will validate
using Zod
before calling
onSubmit

9. Complete Example (All in One)

```

import { useForm } from 'react-hook-form'
import { z } from 'zod'
import { zodResolver } from '@hookform/resolvers/zod'

const schema = z.object({
  name: z.string().min(2, 'Name must be at least 2 characters'),
  email: z.string().email('Invalid email address'),
  age: z.coerce.number().min(18, 'Must be 18 or above'),
  password: z.string().min(6, 'Password must be at least 6 chars'),
  confirm: z.string()
}).refine((data) => data.password === data.confirm, {
  message: 'Passwords do not match',
  path: ['confirm']
})

export default function MyForm() {
  const { register, handleSubmit, formState: { errors } } = useForm({
    resolver: zodResolver(schema)
  })
  const onSubmit = (data) => console.log(data)

  return (
    <form onSubmit={handleSubmit(onSubmit)} noValidate>
      {/* all inputs here using register('field') */}
      <button type="submit">Submit</button>
    </form>
  )
}

```

Zod validates
Automatically
before
submit

Common Input Validations (Zod)

Validation	Example	Meaning
z.string().min(3)	Minimum length	At least 3 characters
z.string().max(10)	Maximum length	At most 10 characters
z.string().email()	Email validation	Must be valid email
z.coerce.number().min(1)	Number + minimum	Must be >= 1
z.enum(['a', 'b'])	Enum validation	Value must be 'a' or 'b'
z.optional()	Optional field	Field is optional
.refine()	Custom validation	Custom rule

Build your
own rules
easily
with Zod



10. Useful RHF + Zod Tips

- ✓ Use `noValidate` on `<form>` to prevent native validation.
- ✓ Always show errors with `errors.field?.message`
- ✓ Use `defaultValues` in `useForm()` for controlled defaults.
- ✓ Use mode: 'onChange' for real-time validation.
- ✓ Zod runs only on submit (or trigger), not on every keypress.

Tip:
Better UX
comes from
good validation
strategy!



11. Default Values

```
const { register, handleSubmit, formState: { errors } } = useForm({
  resolver: zodResolver(schema),
  defaultValues: {
    name: '',
    email: '',
    age: '',
  }
})
```

`defaultValues`
pre-fills form
and prevents
uncontrolled
warnings

12. Modes (Validation Timing)

Mode	When it Validates
onSubmit (default)	When form is <u>submitted</u>
onChange	On every change
onBlur	When input loses focus
all	On change + blur + submit

Use
onChange
for live
validation

13. Trigger Manual Validation

```
const { trigger } = useForm()
// Validate specific field
trigger('email')
// Validate whole form
trigger()
```

Useful for
custom buttons
or conditional
validation

14. Quick Revision

- Install RHF + Zod + Resolver.
- Create Zod schema.
- Use `useForm({ resolver: zodResolver(schema) })`.
- Use `register()` for inputs.
- Show errors using `errors.field?.message`.
- Handle form with `handleSubmit()`.
- Enjoy clean, fast and type-safe forms!



Complete Flow



One Line Summary

React Hook Form + Zod = Fast Forms + Strong Validation + Great Developer Experience ❤️

★ Build Smart Forms, Ship Better Apps! 😊 ★